# Registro de fuentes y uso de IA

Diseño de Arquitecturas Distribuidas

Guato Jairo | AD01 | DISTR-4BM-B1

## Declaración de uso de IA

Utilicé IA generativa para comprender los requisitos de la práctica, organizar el desarrollo de los microservicios y preparar las pruebas en Postman. Este registro reúne las consultas y sus referencias.

Los prompts se reformularon para facilitar su lectura. El de la consulta 5 se reconstruyó a partir del propósito registrado. Las fuentes sugeridas se añadieron durante la revisión. Los números entre corchetes remiten a la bibliografía.

# Consulta 1 Alcance de la práctica

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

5 de noviembre de 2026

**Propósito**

Delimitar los requisitos de la actividad y acordar la función de cada herramienta de IA durante el desarrollo.

**Prompt reformulado**

Una empresa de productos tecnológicos con varias bodegas necesita separar el catálogo comercial del control físico del inventario. La actividad AD01 de Aplicaciones Distribuidas requiere implementar tres microservicios REST independientes con ASP.NET Core Web API y SQL Server. Cada servicio tendrá su propia base de datos.

1. Catalogo.Api (CatalogoDB). Categoria (IdCategoria, Nombre, Descripcion), Marca (IdMarca, Nombre, PaisOrigen) y Producto (IdProducto, IdCategoria, IdMarca, Nombre, SKU, Precio, Estado).

2. Inventario.Api (InventarioDB). Bodega (IdBodega, Nombre, Ciudad, Direccion), Existencia (IdExistencia, IdProducto, IdBodega, Stock), MovimientoInventario (IdMovimiento, IdProducto, IdBodega, Tipo, Cantidad, Fecha) y ReservaStock (IdReserva, IdProducto, IdBodega, Cantidad, Estado).

3. Pedidos.Api (PedidosDB). Cliente (IdCliente, Nombres, Cedula, Correo), Pedido (IdPedido, IdCliente, Fecha, Total, Estado) y DetallePedido (IdDetalle, IdPedido, IdProducto, Cantidad, PrecioUnitario).

Define las claves foráneas físicas y las referencias lógicas, valida los datos de entrada y aplica los códigos HTTP 200, 201, 400 y 404 según corresponda.

Utiliza los puertos HTTP 5101 para Catálogo, 5102 para Inventario y 5103 para Pedidos.

**Respuesta aprovechada**

El resumen organizó la práctica en tres microservicios REST, con una base de datos y un puerto propios para cada servicio. Identificó las reglas de integridad referencial, validación de entradas y respuestas HTTP.

**Fuentes sugeridas**

[4], [5], [14]

**Fuentes verificadas y citadas**

[1], [2], [3]

# Consulta 2 Conceptos de arquitectura y desarrollo

**Herramienta y modelo**

ChatGPT Codex (codex exec) como apoyo para la investigación.

**Fecha**

5 de noviembre de 2026

**Propósito**

Aclarar los conceptos de arquitectura distribuida, persistencia de datos, respuestas HTTP y control de versiones que intervienen en la práctica.

**Prompt reformulado**

1. Explica qué es un microservicio y por qué se utiliza el patrón de base de datos por servicio.

2. Distingue una clave foránea física de una referencia lógica mediante los ejemplos Existencia.IdBodega y Existencia.IdProducto.

3. Explica cuándo usar los códigos HTTP 200, 201, 400 y 404 en una API REST.

4. Describe el enfoque Code First de EF Core y el propósito de las migraciones.

5. Explica por qué se excluyen las carpetas bin y obj del repositorio de GitHub.

**Respuesta aprovechada**

La explicación permitió distinguir la clave foránea de Existencia.IdBodega, que SQL Server comprueba dentro de InventarioDB, de la referencia lógica Existencia.IdProducto, que identifica un producto administrado por Catálogo. También aclaró el significado de los códigos 200, 201, 400 y 404, la evolución del esquema mediante migraciones de EF Core y el uso de .gitignore para excluir bin y obj del repositorio.

**Fuentes sugeridas**

[3], [8], [9]

**Fuentes verificadas y citadas**

[2], [4], [5], [6], [7]

# Consulta 3 Diseño de las bases de datos

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

5 de noviembre de 2026

**Propósito**

Definir las tres bases de datos con los campos requeridos por el enunciado y preparar las cadenas de conexión de los proyectos de la solución de Visual Studio.

**Prompt reformulado**

Crea las tres bases de datos según el enunciado y prepara las cadenas de conexión para vincular cada una con su proyecto en Visual Studio.

Crea únicamente los campos requeridos.

**Respuesta aprovechada**

La respuesta precisó qué tablas corresponden a CatalogoDB, InventarioDB y PedidosDB. Cada proyecto quedó asociado a su propia cadena de conexión en la sección ConnectionStrings de appsettings.json.

**Fuentes sugeridas**

[5], [12], [13]

**Fuentes verificadas y citadas**

[3], [10], [11]

# Consulta 4 Organización de los proyectos

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

6 de noviembre de 2026

**Propósito**

Definir una estructura de carpetas para los tres proyectos como preparación para completar Pedidos.Api, incorporar datos de prueba y revisar la solución.

**Prompt reformulado**

Elabora una guía para organizar las carpetas de cada microservicio. Indica qué carpetas debo crear, en qué orden conviene hacerlo y qué archivos o clases debo colocar en cada una.

**Respuesta aprovechada**

La guía propuso separar las entidades en Models, el contexto de datos en Data y las operaciones HTTP en Controllers. Los proyectos también utilizan DTOs para representar los datos que intercambian por la API. Esta distribución facilita localizar las clases y entender qué responsabilidad tiene cada una.

**Fuentes sugeridas**

[8], [11], [14]

**Fuentes verificadas y citadas**

[2], [3], [6]

# Consulta 5 Configuración de las conexiones y los puertos

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

6 de noviembre de 2026

**Propósito**

Comprender cómo configurar las cadenas de conexión y asignar a cada microservicio su puerto HTTP de desarrollo.

**Prompt reconstruido**

Guíame paso a paso para configurar las cadenas de conexión de las tres bases de datos y asignar el puerto HTTP de cada microservicio. Explica qué archivos debo modificar para ejecutar los tres proyectos desde Visual Studio.

**Respuesta aprovechada**

La guía distinguió dos configuraciones. En appsettings.json, cada cadena de conexión identifica la base de datos que utiliza el servicio. En Properties/launchSettings.json, el perfil HTTP fija la dirección local donde recibe peticiones. Catálogo usa el puerto 5101, Inventario el 5102 y Pedidos el 5103.

**Fuentes sugeridas**

[2], [14], [16]

**Fuentes verificadas y citadas**

[10], [11], [15]

# Consulta 6 Secuencia de implementación

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

6 de noviembre de 2026

**Propósito**

Determinar el orden de trabajo para implementar las operaciones de las API y comprender cómo se relacionan las capas de cada proyecto.

**Prompt reformulado**

Indica en qué orden debo trabajar con las clases de Models, Data y Controllers para implementar las operaciones de la API. Explica por qué conviene seguir esa secuencia.

**Respuesta aprovechada**

La secuencia parte de las entidades en Models. Luego se configura el DbContext en Data para relacionar esas entidades con la base de datos. Por último, se implementan los controladores que reciben las peticiones HTTP y usan el contexto para ejecutar las operaciones.

**Fuentes sugeridas**

[6], [8], [14]

**Fuentes verificadas y citadas**

[2], [3], [11]

# Consulta 7 Implementación inicial de Catálogo

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

6 de noviembre de 2026

**Propósito**

Obtener una guía de implementación por clases para comenzar con Catalogo.Api y aplicar la misma secuencia de trabajo a los demás servicios.

**Prompt reformulado**

Proporciona el código de cada clase en el orden en que debo incorporarlo al proyecto. Comienza por Catalogo.Api.

**Respuesta aprovechada**

La respuesta organizó la implementación alrededor de las entidades Categoria, Marca y Producto, su contexto de datos y los controladores correspondientes. Esta secuencia permitió revisar primero las relaciones entre las entidades y después las operaciones que expone la API.

**Fuentes sugeridas**

[5], [8], [17]

**Fuentes verificadas y citadas**

[2], [11], [14]

# Consulta 8 Pruebas de las API en Postman

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

6 de noviembre de 2026

**Propósito**

Establecer un procedimiento para iniciar los servicios en Visual Studio y probar en Postman las operaciones de Catálogo, Inventario y Pedidos.

**Prompt reformulado**

Elabora una guía paso a paso para probar en Postman todas las operaciones de los tres microservicios.

**Respuesta aprovechada**

La guía indicó cómo seleccionar el método HTTP, copiar la ruta de Swagger y enviar los datos en JSON cuando la operación lo requiera. También orientó la revisión del código de estado y del cuerpo de la respuesta en casos válidos, entradas incorrectas y recursos inexistentes.

**Fuentes sugeridas**

[2], [9], [17]

**Fuentes verificadas y citadas**

[8], [16], [18]

# Consulta 9 Colecciones de pruebas por microservicio

**Herramienta y modelo**

Claude Code (Claude Opus) para la coordinación del proyecto y ChatGPT Codex (codex exec) para la generación de código.

**Fecha**

6 de noviembre de 2026

**Propósito**

Organizar las peticiones de prueba en una colección de Postman por microservicio para poder localizarlas y repetirlas.

**Prompt reformulado**

Explica cómo crear una colección de Postman por cada microservicio y cómo guardar en ella las peticiones de las operaciones implementadas.

**Respuesta aprovechada**

Se organizaron e importaron tres colecciones con un total de 61 peticiones. Cada colección reúne las operaciones del microservicio, sus casos de error y la dirección local que le corresponde. La organización permite abrir y repetir una prueba concreta desde su carpeta.

**Fuentes sugeridas**

[8], [16], [17]

**Fuentes verificadas y citadas**

[18], [19], [20]

# Bibliografía y referencias verificadas


[1] Microsoft. Arquitectura de microservicios de .NET
https://learn.microsoft.com/es-es/dotnet/architecture/microservices/

[2] Microsoft. Creación de API web con ASP.NET Core
https://learn.microsoft.com/es-es/aspnet/core/web-api/?view=aspnetcore-10.0

[3] Microsoft. Información general de Entity Framework Core
https://learn.microsoft.com/es-es/ef/core/

[4] Microsoft. Consideraciones de datos para microservicios
https://learn.microsoft.com/es-es/azure/architecture/microservices/design/data-considerations

[5] Microsoft. Restricciones entre claves principales y claves externas
https://learn.microsoft.com/es-es/sql/relational-databases/tables/primary-and-foreign-key-constraints

[6] Microsoft. Descripción general de las migraciones de EF Core
https://learn.microsoft.com/es-es/ef/core/managing-schemas/migrations/

[7] Proyecto Git. Documentación de gitignore
https://git-scm.com/docs/gitignore

[8] MDN Web Docs. Códigos de estado de respuesta HTTP
https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status

[9] Fielding, R., Nottingham, M., y Reschke, J. (eds.), 2022. HTTP Semantics (RFC 9110)
https://www.rfc-editor.org/rfc/rfc9110.html

[10] Microsoft. Cadenas de conexión en EF Core
https://learn.microsoft.com/es-es/ef/core/miscellaneous/connection-strings

[11] Microsoft. Duración, configuración e inicialización de DbContext
https://learn.microsoft.com/es-es/ef/core/dbcontext-configuration/

[12] Microsoft. Transacciones en EF Core
https://learn.microsoft.com/es-es/ef/core/saving/transactions

[13] Microsoft. Uso de IHttpClientFactory
https://learn.microsoft.com/es-es/dotnet/core/extensions/httpclient-factory

[14] Microsoft. Validación de modelos en ASP.NET Core MVC
https://learn.microsoft.com/es-es/aspnet/core/mvc/models/validation?view=aspnetcore-10.0

[15] Microsoft. Configuración de puntos de conexión de Kestrel
https://learn.microsoft.com/es-es/aspnet/core/fundamentals/servers/kestrel/endpoints?view=aspnetcore-10.0

[16] Microsoft. Establecimiento de varios proyectos de inicio en Visual Studio
https://learn.microsoft.com/es-es/visualstudio/ide/how-to-set-multiple-startup-projects?view=vs-2022

[17] Postman. Introducción a Postman
https://learning.postman.com/docs/getting-started/overview/

[18] Postman. Envío de peticiones con el cliente de Postman
https://learning.postman.com/docs/use/send-requests/create-requests/request-basics

[19] Postman. Creación de colecciones de peticiones
https://learning.postman.com/docs/use/use-collections/create-collections

[20] Postman. Importación de datos en Postman
https://learning.postman.com/docs/getting-started/importing-and-exporting/importing-data/